Experiment 2: Depth First Search (DFS)

Aim:

To implement Depth First Search algorithm to traverse a graph by exploring as deep as possible along each branch before backtracking.

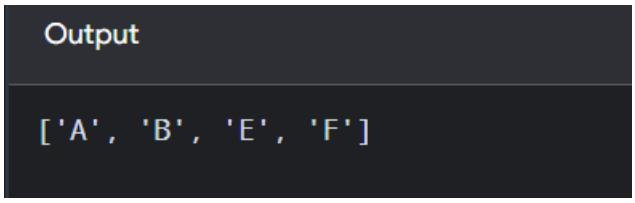
Algorithm:

1. Start from the source node and mark it visited.
2. Recursively visit the first unvisited neighbor.
3. If a dead-end is reached, backtrack to the previous node.
4. Continue until the goal is found or all nodes are explored.
5. Return the path if goal is reached.

Code:

```
def dfs(graph, start, goal, path=None, visited=None):
    if path is None: path = [start]
    if visited is None: visited = set()
    if start == goal:
        return path
    visited.add(start)
    for n in graph.get(start, []):
        if n not in visited:
            res = dfs(graph, n, goal, path + [n], visited)
            if res:
                return res
    return None
graph = {'A':['B','C'], 'B':['D','E'], 'C':['F'], 'D':[], 'E':['F'],
'F':[]}
print("DFS Path:", dfs(graph, 'A', 'F'))
```

Sample Output:



Result:

DFS explored deep into the graph and found a valid path from A to F. Unlike BFS, the path is not guaranteed to be the shortest.